



Subquery

A **subquery** is a SQL query written inside another query. It is used to return intermediate results that the outer query depends on.

Subqueries are commonly used for:

- filtering data dynamically
- comparing values against aggregates
- creating temporary result sets
- simplifying complex logic

They can be used in:

- WHERE clause
- SELECT clause
- FROM clause

1. Base Data View

Short Notes on Each Script Section

1. Database Selection

```
SELECT *  
FROM employees;
```

What it does:

- Displays all employee data
- Used as baseline data for CASE logic



2. Subquery in WHERE Clause (Filter Employees by Department)

```
SELECT *  
FROM employees  
WHERE employee_id IN (  
    SELECT employee_id  
    FROM employee_positions  
    WHERE role_id IN (  
        SELECT role_id  
        FROM job_roles  
    )  
);
```

What it does:

- Finds employees linked to job roles
- Uses nested subqueries to filter employees indirectly
- Demonstrates filtering using derived ID lists

Simplified Business Version (TechSphere Department Filter)

```
SELECT *  
FROM employees  
WHERE department_id IN (  
    SELECT department_id  
    FROM departments  
    WHERE department_name = 'Data Analytics'  
);
```

What it does:

- Returns employees in the Data Analytics department
- Uses subquery instead of JOIN



3. Incorrect Subquery Example (Multi-column Error Case)

```
SELECT *  
FROM employees  
WHERE employee_id IN (  
    SELECT employee_id, department_id  
    FROM employee_positions  
);
```

What happens:

- This query fails

Why:

- IN only accepts **one column**
- Subquery returns multiple columns

Key Lesson:

Subqueries used with IN must return a single column only

4. Subquery in SELECT Clause (Global Average Comparison)

```
SELECT  
    first_name,  
    last_name,  
    (  
        SELECT AVG(base_salary)  
        FROM job_roles  
    ) AS average_salary  
FROM employees;
```

What it does:

- Shows each employee
- Adds company-wide average salary
- Same value repeated for all rows

**Key Idea:**

- Subquery returns a single scalar value
- Used for comparison or benchmarking

5. Incorrect Aggregation Without Subquery

```
SELECT first_name, base_salary, AVG(base_salary)
FROM job_roles
GROUP BY first_name, base_salary;
```

What happens:

- Produces grouped averages per row combination
- Not useful for global comparison

Key Lesson:

- Aggregates need correct grouping OR subquery approach

6. Subquery in FROM Clause (Derived Table)

```
SELECT *
FROM (
    SELECT
        d.department_name,
        COUNT(e.employee_id) AS total_employees
    FROM employees e
    JOIN departments d
        ON e.department_id = d.department_id
    GROUP BY d.department_name
) AS dept_summary;
```

What it does:

- Creates a temporary table (dept_summary)
- Shows number of employees per department

**Key Idea:**

- Subquery acts like a virtual table

7. Using Derived Table for Further Analysis

```
SELECT department_name, total_employees
FROM (
    SELECT
        d.department_name,
        COUNT(e.employee_id) AS total_employees
    FROM employees e
    JOIN departments d
        ON e.department_id = d.department_id
    GROUP BY d.department_name
) AS dept_summary
WHERE total_employees > 1;
```

What it does:

- Filters departments with more than 1 employee
- Demonstrates layered subquery analysis

KEY RULES OF SUBQUERIES

- Must return compatible structure based on usage
- IN subqueries → single column only
- FROM subqueries → must have alias
- Scalar subqueries → return single value
- Can be nested multiple levels

BUSINESS USE CASES IN TECHSPHERE

Subqueries are used for:

- HR analytics (employee filtering)
- salary benchmarking
- department reporting
- KPI calculations
- dashboard data preparation

**FINAL INSIGHT**

Subqueries allow SQL to work in layers, where one query produces data that another query uses, enabling powerful and flexible business analysis without complex joins.